Regression Model Training Notebook

Setup Environment

```
# DO NOT MODIFY THE CODE IN THIS CELL
!pip install -q utstd

from utstd.folders import *
from utstd.ipyrenders import *

at = AtFolder(
    course_code=36106,
    assignment="AT3",
)
at.run()

import warnings
warnings.simplefilter(action='ignore')
```

```
ERROR: pip's dependency resolver does not currently take into account 
umap-learn 0.5.12 requires scikit-learn>=1.6, but you have scikit-lear
hdbscan 0.8.42 requires scikit-learn>=1.6, but you have scikit-learn 1
Drive already mounted at /content/gdrive; to attempt to forcibly remou

You can now save your data files in: /content/gdrive/MyDrive/36106/ass
```

Student Information

```
group_name = "36106-26AU-AT3-Group 20"
student_name = "Devvrat Charusmiti Joshi"
student_id = "25657887"
```

```
# Do not modify this code
print_tile(size="h1", key='group_name', value=group_name)
```

group_name

36106-26AU-AT3-Group 20

```
# Do not modify this code
print_tile(size="h1", key='student_name', value=student_name)
```

student_name

Devvrat Charusmiti Joshi

```
# Do not modify this code
print_tile(size="h1", key='student_id', value=student_id)
```

student_id

25657887

---

0. Python Packages

0.a Install Additional Packages

> If you are using additional packages, you need to install them here using the command: `! pip install <package_name>`

0.b Import Packages

```
import pandas as pd
import altair as alt
import numpy as np


pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", 100)

alt.data_transformers.disable_max_rows()

from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error,
```

B. Business Understanding

```
business_use_case_description = """
The business use case is to predict the total sales amount of an orde
This regression model uses order-level transaction, customer, person,
The baseline model predicted the same average order value for every o
"""
```

```
# Do not modify this code
print_tile(size="h3", key='business_use_case_description', value=busi
```

business_use_case_description

The business use case is to predict the total sales amount of an order for a global retailer. The target variable is order_total, which represents the total monetary value of each sales order. This regression model uses order-level transaction, customer, person, territory, channel, and time-based features prepared in the data preparation notebook. The aim is to support revenue forecasting, sales planning, business reporting, and early identification of

```
business_objectives = """
The main business objective is to estimate order value more accurately
If the model predicts order_total accurately, the business can better
Incorrect predictions may create business risks. Underestimating orde
"""
```

```
# Do not modify this code
print_tile(size="h3", key='business_objectives', value=business_objec
```

business_objectives

The main business objective is to estimate order value more accurately than the baseline model. Accurate predictions can help the retailer improve revenue forecasting, inventory planning, campaign planning, and territory-level sales analysis. If the model predicts order_total accurately, the business can better understand expected sales performance and identify orders or customer segments that are likely to contribute higher value.

```
stakeholders_expectations_explanations = """
The main users of the predictions are business analysts, sales manage
Sales managers can use predicted order values to understand expected
The predictions should be interpreted as estimated order values, not
"""
```

```
# Do not modify this code
print_tile(size="h3", key='stakeholders_expectations_explanations', v
```

stakeholders_expectations_explanations

The main users of the predictions are business analysts, sales managers, marketing teams, and inventory or planning teams. Sales managers can use predicted order values to understand expected revenue trends. Marketing teams can use the predictions to identify higher-value order patterns and improve campaign planning. Inventory and planning teams

---

C. Data Understanding

C.1 Load Datasets

```
X_train = pd.read_csv(at.folder_path / "X_train.csv")
X_val = pd.read_csv(at.folder_path / "X_val.csv")
X_test = pd.read_csv(at.folder_path / "X_test.csv")

y_train = pd.read_csv(at.folder_path / "y_train.csv")
y_val = pd.read_csv(at.folder_path / "y_val.csv")
y_test = pd.read_csv(at.folder_path / "y_test.csv")

target_column = "order_total"

y_train_model = y_train[target_column]
y_val_model = y_val[target_column]
y_test_model = y_test[target_column]

print("X_train shape:", X_train.shape)
print("X_val shape:", X_val.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train_model.shape)
print("y_val shape:", y_val_model.shape)
print("y_test shape:", y_test_model.shape)
```

```
X_train shape: (10749, 39)
X_val shape: (2304, 39)
X_test shape: (2304, 39)
y_train shape: (10749,)
y_val shape: (2304,)
y_test shape: (2304,)
```

C.2 Define Target variable

```
target_column = "order_total"

target_summary = pd.DataFrame({
    "dataset": ["Training", "Validation", "Testing"],
    "rows": [len(y_train), len(y_val), len(y_test)],
    "target_column": [target_column, target_column, target_column],
    "mean_order_total": [
        y_train[target_column].mean(),
        y_val[target_column].mean(),
        y_test[target_column].mean()
    ],
    "median_order_total": [
        y_train[target_column].median(),
        y_val[target_column].median(),
        y_test[target_column].median()
    ]
})

target_summary
```

	dataset	rows	target_column	mean_order_total	median_order_total
0	Training	10749	order_total	2253.428325	1174.480
1	Validation	2304	order_total	1536.918902	588.960
2	Testing	2304	order_total	1184.834210	285.222

```
target_definition_explanations = """
The target variable is order_total. It represents the total sales amo
This target is appropriate for the selected business use case because
The target is stored separately in y_train, y_val, and y_test to avoi
"""
```

```
# Do not modify this code
print_tile(size="h3", key='target_definition_explanations', value=tar
```

target_definition_explanations

The target variable is order_total. It represents the total sales amount of a sales order and was created in the preparation notebook by aggregating line_total from the sales_order_detail dataset at the sales_order_id level. This target is appropriate for the selected business use case because the business wants to predict a continuous monetary value. Since order_total is numerical and continuous, the problem is formulated as a supervised

C.3 Create Target variable

```
target_name = target_column

y_train_model = y_train[target_column]
y_val_model = y_val[target_column]
y_test_model = y_test[target_column]

print("Target name:", target_name)
print("y_train_model shape:", y_train_model.shape)
print("y_val_model shape:", y_val_model.shape)
print("y_test_model shape:", y_test_model.shape)
```

```
Target name: order_total
y_train_model shape: (10749,)
y_val_model shape: (2304,)
y_test_model shape: (2304,)
```

C.4 Explore Target variable

```
target_distribution = pd.DataFrame({
    "dataset": ["Training", "Validation", "Testing"],
    "mean": [
        y_train_model.mean(),
```

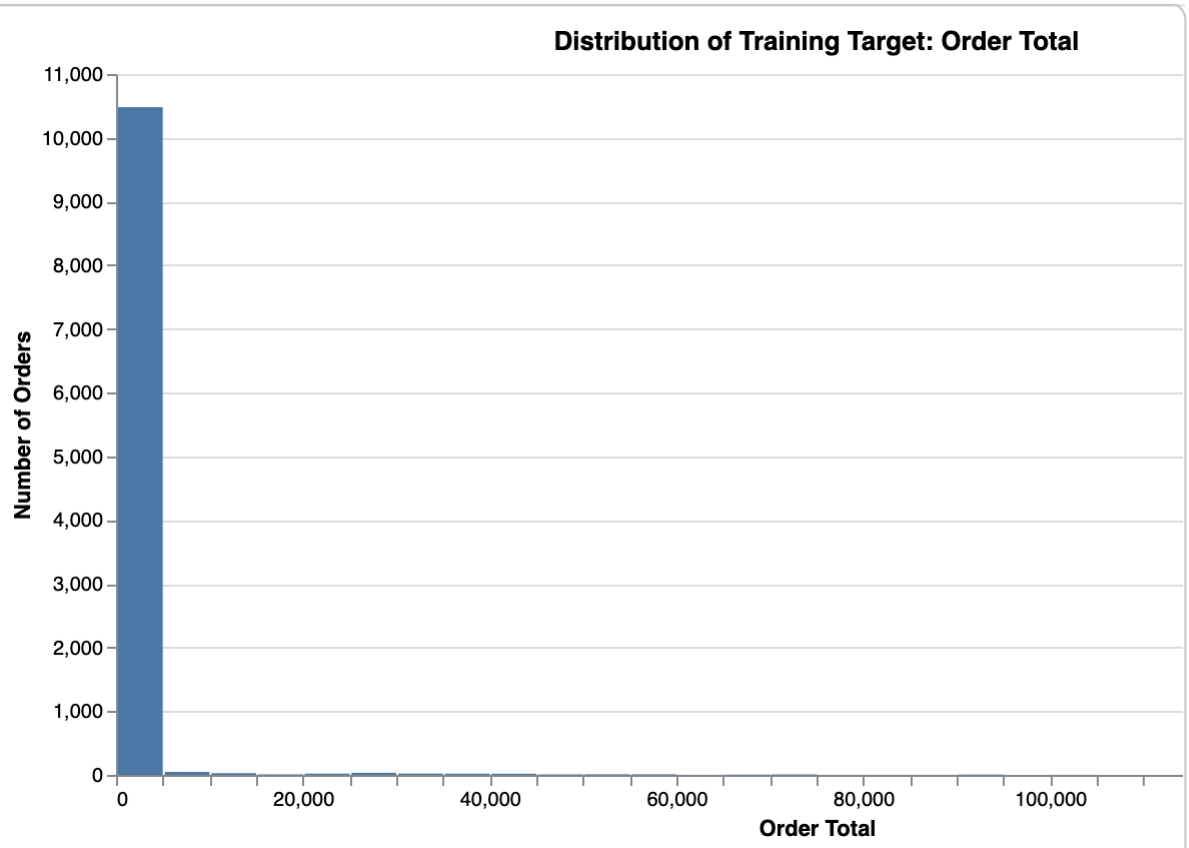
```
        y_val_model.mean(),
        y_test_model.mean()
    ],
    "median": [
        y_train_model.median(),
        y_val_model.median(),
        y_test_model.median()
    ],
    "min": [
        y_train_model.min(),
        y_val_model.min(),
        y_test_model.min()
    ],
    "max": [
        y_train_model.max(),
        y_val_model.max(),
        y_test_model.max()
    ],
    "std": [
        y_train_model.std(),
        y_val_model.std(),
        y_test_model.std()
    ]
})

target_distribution
```

	dataset	mean	median	min	max	std
0	Training	2253.428325	1174.480	2.29	147390.932828	7011.736213
1	Validation	1536.918902	588.960	2.29	112312.554000	5923.916686
2	Testing	1184.834210	285.222	2.29	89869.276314	4467.320667

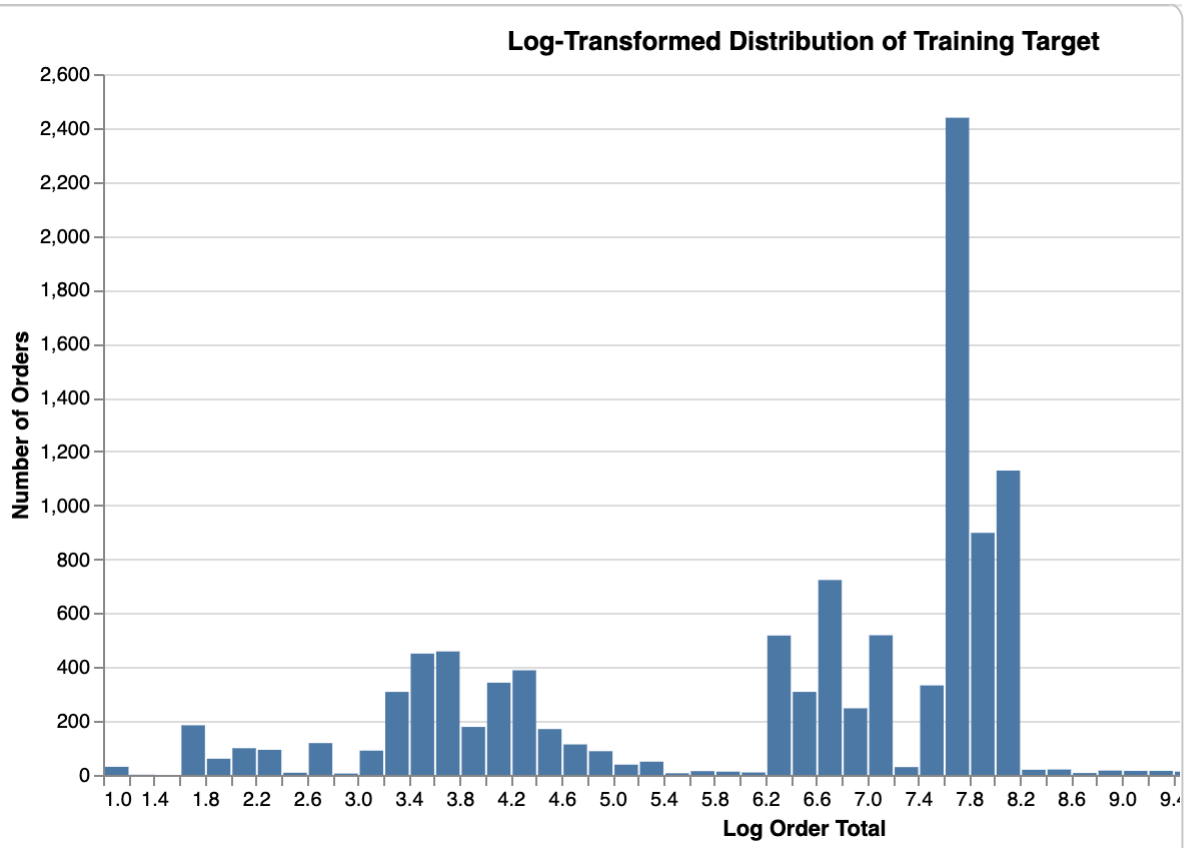
```
target_plot_data = pd.DataFrame({
    "order_total": y_train_model,
    "dataset": "Training"
})

alt.Chart(target_plot_data).mark_bar().encode(
    x=alt.X(
        "order_total:Q",
        bin=alt.Bin(maxbins=60),
        title="Order Total"
    ),
    y=alt.Y("count():Q", title="Number of Orders"),
    tooltip=["count()"]
).properties(
    title="Distribution of Training Target: Order Total",
    width=700,
    height=350
)
```




```
log_target_plot_data = target_plot_data.copy()
log_target_plot_data["log_order_total"] = np.log1p(log_target_plot_da

alt.Chart(log_target_plot_data).mark_bar().encode(
    x=alt.X(
        "log_order_total:Q",
        bin=alt.Bin(maxbins=60),
        title="Log Order Total"
    ),
    y=alt.Y("count():Q", title="Number of Orders"),
    tooltip=["count()"]
).properties(
    title="Log-Transformed Distribution of Training Target",
    width=700,
    height=350
)
```




```
target_distribution_explanations = """
The target variable order_total is right-skewed. Most orders have rela
The validation and testing target distributions have lower average or
The skewness of the target is important for model evaluation. MAE is
"""
```

```
# Do not modify this code
print_tile(size="h3", key='target_distribution_explanations', value=t
```

target_distribution_explanations

The target variable order_total is right-skewed. Most orders have relatively low or moderate sales values, while a smaller number of orders have very high values. This is visible from the difference between the mean and median order values. The validation and testing target distributions have lower average order totals than the training set. This is expected because the preparation notebook used a time-based split. A time-based split better represents a real business situation where earlier orders are used to

C.5 Explore Feature of Interest `total_quantity`

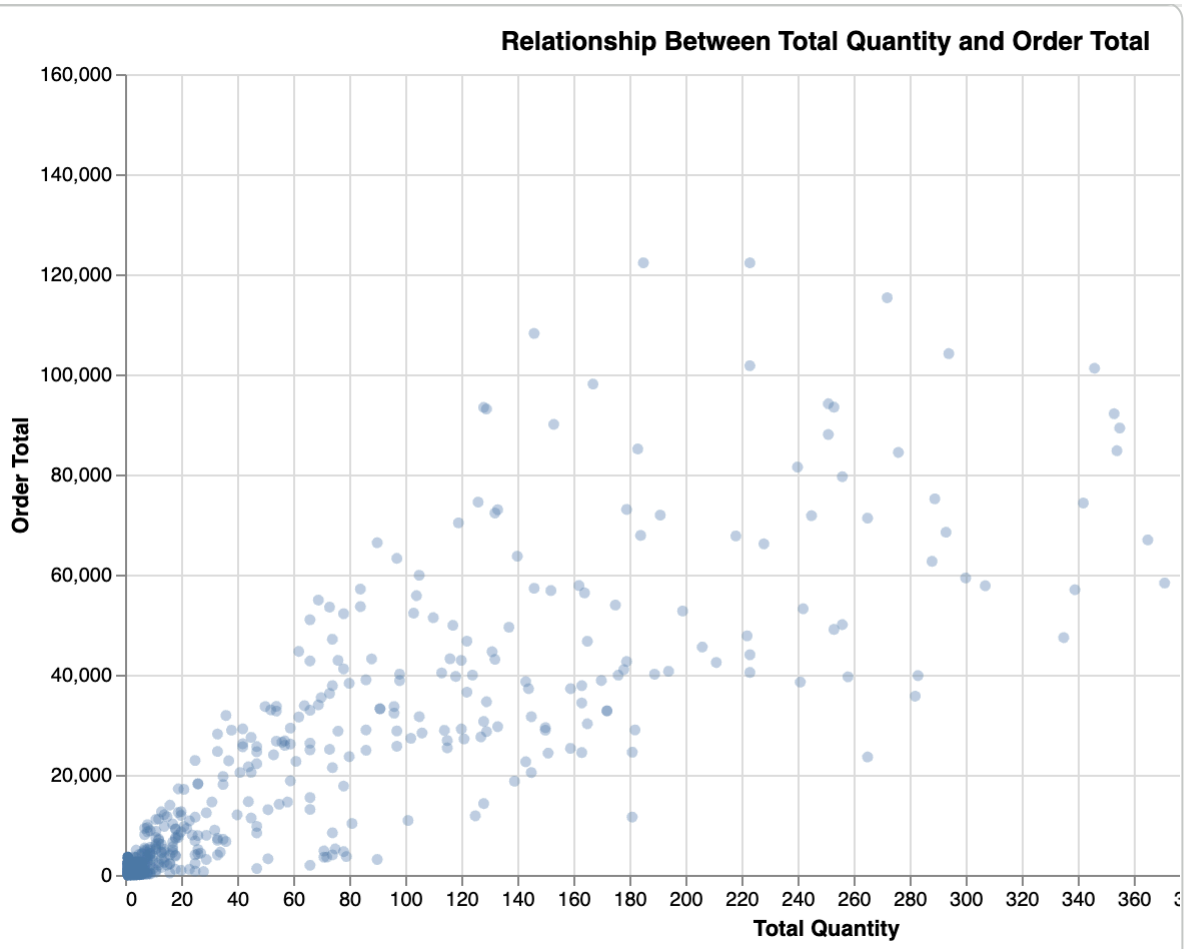
```
feature_1 = "total_quantity"

feature_1_summary = X_train[feature_1].describe().to_frame(name=featu
feature_1_summary
```

	total_quantity
count	10749.000000
mean	4.900549
std	23.004604
min	1.000000
25%	1.000000
50%	2.000000
75%	3.000000
max	476.000000

```
feature_1_plot_data = pd.DataFrame({
    feature_1: X_train[feature_1],
    "order_total": y_train_model
})

alt.Chart(feature_1_plot_data).mark_circle(opacity=0.35).encode(
    x=alt.X(f"{feature_1}:Q", title="Total Quantity"),
    y=alt.Y("order_total:Q", title="Order Total"),
    tooltip=[feature_1, "order_total"]
).properties(
    title="Relationship Between Total Quantity and Order Total",
    width=700,
    height=400
)
```




```
feature_1_insights = """
The total_quantity feature represents the total number of units purcha
However, total_quantity alone cannot fully explain order_total becaus
"""
```

```
# Do not modify this code
print_tile(size="h3", key='feature_1_insights', value=feature_1_insigl
```

feature_1_insights

The total_quantity feature represents the total number of units purchased in an order. It is a useful feature for predicting order_total because orders with higher quantities are generally expected to have higher total sales values. However, total_quantity alone cannot fully explain order_total because order value also depends on product prices, discounts, product

C.6 Explore Feature of Interest `average_unit_price`

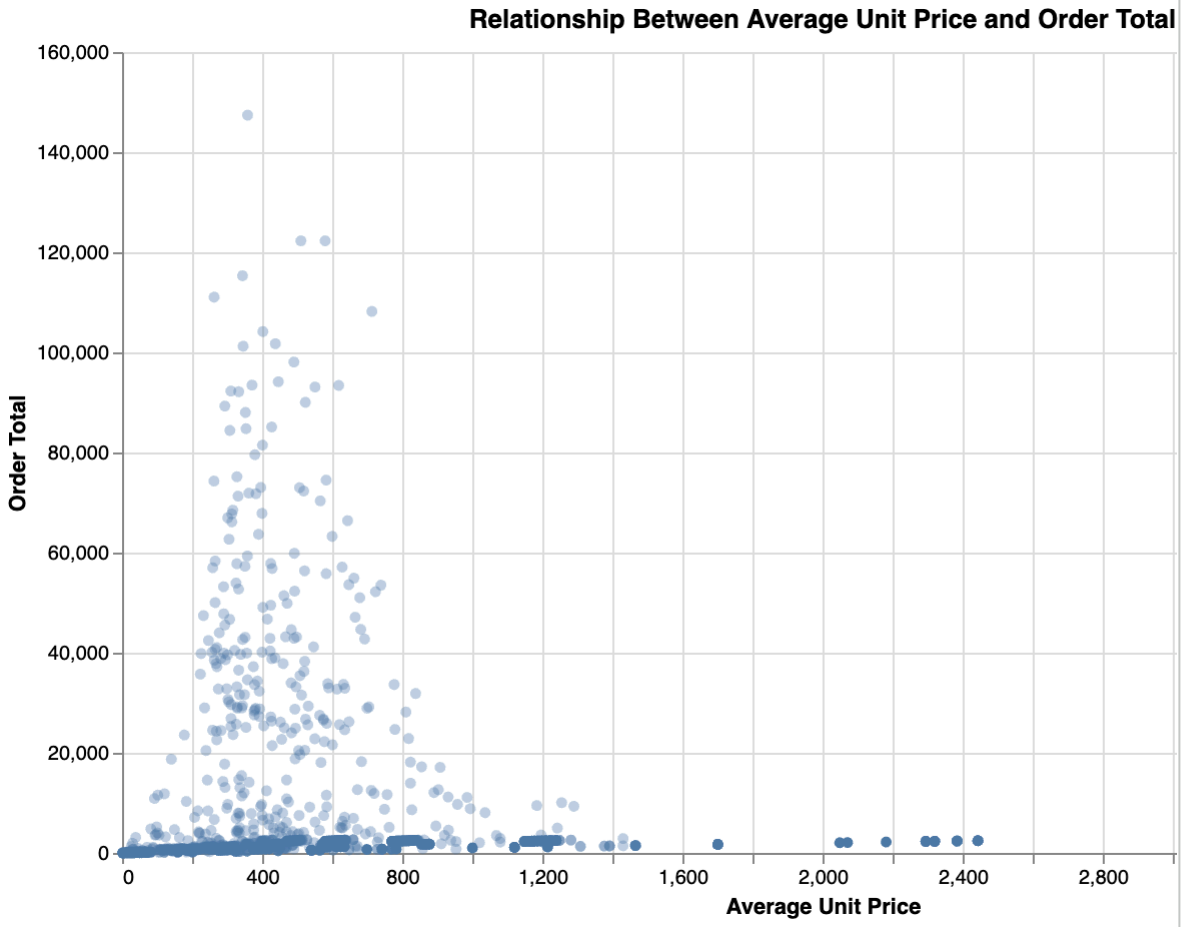
```
feature_2 = "average_unit_price"

feature_2_summary = X_train[feature_2].describe().to_frame(name=featu
feature_2_summary
```

	average_unit_price
count	10749.000000
mean	1024.342008
std	1142.473531
min	1.374000
25%	31.656667
50%	592.492500
75%	2049.098200
max	3578.270000

```
feature_2_plot_data = pd.DataFrame({
    feature_2: X_train[feature_2],
    "order_total": y_train_model
})

alt.Chart(feature_2_plot_data).mark_circle(opacity=0.35).encode(
    x=alt.X(f"{feature_2}:Q", title="Average Unit Price"),
    y=alt.Y("order_total:Q", title="Order Total"),
    tooltip=[feature_2, "order_total"]
).properties(
    title="Relationship Between Average Unit Price and Order Total",
    width=700,
    height=400
)
```




```
feature_2_insights = """
The average_unit_price feature represents the average product price w
Orders with higher average unit prices are more likely to have higher
"""
```

```
# Do not modify this code
print_tile(size="h3", key='feature_2_insights', value=feature_2_insig
```

feature_2_insights

The average_unit_price feature represents the average product price within an order. This is an important predictor because the EDA showed that unit price has a strong relationship with sales amount. Orders with higher average unit prices are more likely to have higher order totals. However

C.7 Explore Feature of Interest `product_diversity_ratio`

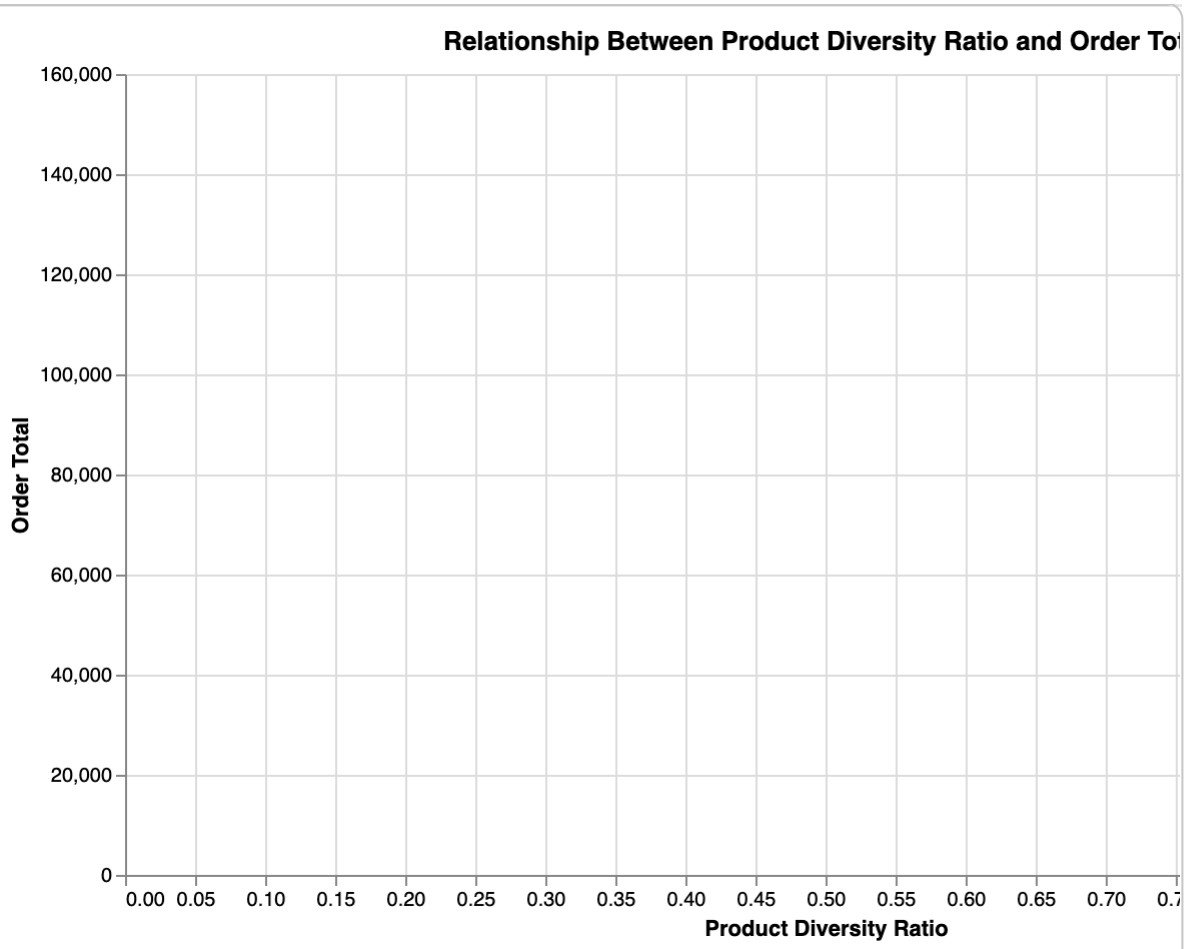
```
feature_3 = "product_diversity_ratio"

feature_3_summary = X_train[feature_3].describe().to_frame(name=featu
feature_3_summary
```

	product_diversity_ratio
count	10749.0
mean	1.0
std	0.0
min	1.0
25%	1.0
50%	1.0
75%	1.0
max	1.0

```
feature_3_plot_data = pd.DataFrame({
    feature_3: X_train[feature_3],
    "order_total": y_train_model
})

alt.Chart(feature_3_plot_data).mark_circle(opacity=0.35).encode(
    x=alt.X(f"{feature_3}:Q", title="Product Diversity Ratio"),
    y=alt.Y("order_total:Q", title="Order Total"),
    tooltip=[feature_3, "order_total"]
).properties(
    title="Relationship Between Product Diversity Ratio and Order Tot
    width=700,
    height=400
)
```




```
feature_n_insights = """
The product_diversity_ratio feature measures the variety of products
This feature may help explain purchasing behaviour, but it may not be
"""
```

```
# Do not modify this code
print_tile(size="h3", key='feature_n_insights', value=feature_n_insig
```

feature_n_insights

The product_diversity_ratio feature measures the variety of products in an order relative to the number of line items. It helps distinguish between orders containing different products and orders containing repeated or less varied product lines. This feature may help explain purchasing behaviour

C.n Explore Feature of Interest `\<put feature name here\>`

> You can add more cells related to other feeatures in this section

---

D. Feature Selection

```
features_list = X_train.columns.tolist()

feature_selection_summary = pd.DataFrame({
    "feature": features_list,
    "data_type": X_train.dtypes.astype(str).values,
    "missing_values": X_train.isna().sum().values
})

print("Number of selected features:", len(features_list))
feature_selection_summary
```

```
Number of selected features: 39
```

	feature	data_type	missing_values
0	number_of_line_items	int64	0
1	total_quantity	float64	0
2	average_quantity	float64	0
3	average_unit_price	float64	0
4	max_unit_price	float64	0
5	min_unit_price	float64	0
11	unique_special_offers	int64	0
12	order_year	int64	0
13	order_month	int64	0
14	order_dayofweek	int64	0
15	is_weekend	int64	0
16	has_discount	int64	0
17	product_diversity_ratio	float64	0
18	unit_price_range	float64	0
19	special_offer_diversity_ratio	float64	0
29	territory_id_0ad4c625-bb65-4376-8a41-0a65719b0db8	bool	0
30	territory_id_25d73ad2-9e13-410b-8b0d-5f26e2b9e4f2	bool	0

```
feature_selection_explanations = """
The selected features are the prepared input features from the data p
The selected features include transaction-based features such as quan
"""
```

```
# Do not modify this code
print_tile(size="h3", key='feature_selection_explanations', value=fea
```

feature_selection_explanations

The selected features are the prepared input features from the data preparation notebook. The target variable order_total is stored separately in y_train, y_val, and y_test, so it is not included in the feature set. The selected features include transaction-based features such as quantity.

E. Data Preparation

E.1 Data Transformation

```
missing_value_summary = pd.DataFrame({
    "dataset": ["X_train", "X_val", "X_test", "y_train", "y_val", "y_
    "missing_values": [
        X_train.isna().sum().sum(),
        X_val.isna().sum().sum(),
        X_test.isna().sum().sum(),
        y_train.isna().sum().sum(),
        y_val.isna().sum().sum(),
        y_test.isna().sum().sum()
    ]
})

missing_value_summary
```

	dataset	missing_values
0	X_train	0
1	X_val	0
2	X_test	0
3	y_train	0
4	y_val	0
5	y_test	0

31	territory_id_2ac923e9-3043-4f5e-bae0-ad28089bf187	bool	0
32	territory_id_e0e3bac4-790e-4617-84b3-be0c2bdb7070	bool	0
33	person_type_IN	bool	0
34	person_type_SC	bool	0
35	son_type_Unknown	bool	0
36	email_promotion_0.0	bool	0

```
data_cleaning_1_explanations = """
A missing value check is performed before model training to ensure tha
The prepared datasets should have no missing values because missing va
"""
```

```
# Do not modify this code
print_tile(size="h3", key='data_cleaning_1_explanations', value=data_
```

data_cleaning_1_explanations

A missing value check is performed before model training to ensure that the regression algorithm receives complete input data. Missing values can cause model fitting to fail or produce unreliable results. The prepared datasets should have no missing values because missing value handling

E.2 Data Transformation

```
non_numeric_columns = X_train.select_dtypes(exclude=[np.number]).colu

numeric_type_check = pd.DataFrame({
    "check": ["Number of non-numeric feature columns"],
    "value": [len(non_numeric_columns)]
})

print("Non-numeric columns:", non_numeric_columns)
numeric_type_check
```

```
Non-numeric columns: ['online_order_flag_0.0', 'online_order_flag_1.0'
```

	check	value
0	Number of non-numeric feature columns	13

```
# Convert boolean columns to integer if any exist

for dataset in [X_train, X_val, X_test]:
    bool_columns = dataset.select_dtypes(include=["bool"]).columns
    dataset[bool_columns] = dataset[bool_columns].astype(int)
```

```
print("Boolean columns converted to integer where required.")
```

```
Boolean columns converted to integer where required.
```

```
data_cleaning_2_explanations = """
Random Forest Regressor requires numerical input features. Since cate
This step checks for any remaining non-numeric columns and converts b
"""
```

```
# Do not modify this code
print_tile(size="h3", key='data_cleaning_2_explanations', value=data_
```

data_cleaning_2_explanations

Random Forest Regressor requires numerical input features. Since categorical variables were one-hot encoded in the preparation notebook, all feature columns should already be numeric. This step checks for any remaining non-numeric columns and converts boolean columns to integer

E.3 Data Transformation

```
# Ensure validation and testing columns match training columns exactl

X_val = X_val.reindex(columns=X_train.columns, fill_value=0)
X_test = X_test.reindex(columns=X_train.columns, fill_value=0)

column_alignment_summary = pd.DataFrame({
    "dataset": ["X_train", "X_val", "X_test"],
    "columns": [X_train.shape[1], X_val.shape[1], X_test.shape[1]],
    "same_columns_as_training": [
        True,
        list(X_val.columns) == list(X_train.columns),
        list(X_test.columns) == list(X_train.columns)
    ]
})

column_alignment_summary
```

	dataset	columns	same_columns_as_training
0	X_train	39	True
1	X_val	39	True
2	X_test	39	True

```
data_cleaning_3_explanations = """
The validation and testing feature columns are aligned to the trainin
"""
```

```
# Do not modify this code
print_tile(size="h3", key='data_cleaning_3_explanations', value=data_
```

data_cleaning_3_explanations

The validation and testing feature columns are aligned to the training feature columns. This is important because machine learning models require the same feature structure during training and prediction.

E.n Fixing "<describe_issue_here>"

> You can add more cells related to other issues in this section

```
Start coding or generate with AI.
```

F. Feature Engineering

F.1 New Feature "<put_name_here>"

```
# No additional feature engineering is applied in this notebook.
# The model uses the prepared features created in the Preparation not

X_train_fe = X_train.copy()
X_val_fe = X_val.copy()
X_test_fe = X_test.copy()

print("X_train_fe shape:", X_train_fe.shape)
print("X_val_fe shape:", X_val_fe.shape)
print("X_test_fe shape:", X_test_fe.shape)
```

```
X_train_fe shape: (10749, 39)
X_val_fe shape: (2304, 39)
X_test_fe shape: (2304, 39)
```

```
feature_engineering_1_explanations = """
No additional feature engineering is applied in this regression noteb
This makes the Random Forest Regression result easier to compare agai
"""
```

```
# Do not modify this code
print_tile(size="h3", key='feature_engineering_1_explanations', value=
```

feature_engineering_1_explanations

No additional feature engineering is applied in this regression notebook. The prepared features from the Preparation notebook are used directly to keep this experiment consistent with the baseline and the data preparation workflow. This makes the Random Forest Regression result easier to

F.2 New Feature "<put_name_here>"

```
# Check final feature columns used for modelling

feature_columns_used = pd.DataFrame({
    "feature": X_train_fe.columns,
    "data_type": X_train_fe.dtypes.astype(str).values
})

feature_columns_used
```

	feature	data_type
0	number_of_line_items	int64
1	total_quantity	float64
2	average_quantity	float64
3	average_unit_price	float64
4	max_unit_price	float64
5	min_unit_price	float64
8	max_discount	float64
11	unique_special_offers	int64
12	order_year	int64
13	order_month	int64
14	order_dayofweek	int64
15	is_weekend	int64
16	has_discount	int64
17	product_diversity_ratio	float64
18	unit_price_range	float64
24	log_std_unit_price	float64
25	log_unit_price_range	float64
26	online_order_flag_0.0	int64
27	online_order_flag_1.0	int64
28	status_5.0	int64
29	territory_id_0ad4e625-5b65-4476-8a4f-0a65719b0db8	int64
30	territory_id_25d73ad2-9e13-410b-8b0d-5f26e2b9e4f2	int64
31	territory_id_2ac923e9-3043-4f5e-bae0-ad28089bf187	int64

```
feature_engineering_2_explanations = """
This section verifies the final set of prepared features used by the m
"""
```

```
# Do not modify this code
print_tile(size="h3", key='feature_engineering_2_explanations', value=
```

feature_engineering_2_explanations

This section verifies the final set of prepared features used by the model. The features include transaction, price, discount, product variety, special offer, territory, channel, person, and time-based variables created during

F.3 New Feature "<put_name_here>"

Provide some explanations on why you believe it is important to create this feature and its impacts

```
# <Student to fill this section>
feature_engineering_n_explanations = """
Provide some explanations on why you believe it is important to creat
"""
```

```
# Do not modify this code
print_tile(size="h3", key='feature_engineering_n_explanations', value=
```

F.n Fixing "<describe_issue_here>"

You can add more cells related to new features in this section

G. Data Preparation for Modeling

32	territory_id_e0e3bac4-790e-4617-84b3-be0c2bdb7070	int64
33	person_type_IN	int64

G.1 Split Datasets

```
split_summary = pd.DataFrame({
    "dataset": ["Training", "Validation", "Testing"],
    "X_rows": [len(X_train), len(X_val), len(X_test)],
    "y_rows": [len(y_train_model), len(y_val_model), len(y_test_model
    "features": [X_train.shape[1], X_val.shape[1], X_test.shape[1]]
})

split_summary
```

	dataset	X_rows	y_rows	features
0	Training	10749	10749	39
1	Validation	2304	2304	39
2	Testing	2304	2304	39

```
data_splitting_explanations = """
The datasets were already split in the preparation notebook using a t
"""
```

```
# Do not modify this code
print_tile(size="h3", key='data_splitting_explanations', value=data_s
```

data_splitting_explanations

The datasets were already split in the preparation notebook using a time-based split. This means earlier orders were used for training, later orders for validation, and the most recent orders for testing.

G.2 Data Transformation "Final Feature Check"

```
final_feature_matrix_check = pd.DataFrame({
    "dataset": ["X_train", "X_val", "X_test"],
    "rows": [X_train.shape[0], X_val.shape[0], X_test.shape[0]],
    "columns": [X_train.shape[1], X_val.shape[1], X_test.shape[1]],
    "missing_values": [
        X_train.isna().sum().sum(),
        X_val.isna().sum().sum(),
        X_test.isna().sum().sum()
    ],
    "non_numeric_columns": [
        len(X_train.select_dtypes(exclude=[np.number]).columns),
        len(X_val.select_dtypes(exclude=[np.number]).columns),
        len(X_test.select_dtypes(exclude=[np.number]).columns)
    ]
})

final_feature_matrix_check
```

	dataset	rows	columns	missing_values	non_numeric_columns
0	X_train	10749	39	0	0
1	X_val	2304	39	0	0
2	X_test	2304	39	0	0

```
data_transformation_1_explanations = """
The final feature matrix check confirms that the training, validation
"""
```

```
# Do not modify this code
print_tile(size="h3", key='data_transformation_1_explanations', value=
```

data_transformation_1_explanations

The final feature matrix check confirms that the training, validation, and testing feature datasets have the same structure, no missing values, and no non-numeric columns.

G.3 Data Transformation "Target Shape Check"

```
target_shape_check = pd.DataFrame({
    "dataset": ["y_train", "y_val", "y_test"],
    "rows": [len(y_train_model), len(y_val_model), len(y_test_model)]
    "missing_values": [
        y_train_model.isna().sum(),
        y_val_model.isna().sum(),
        y_test_model.isna().sum()
    ],
    "mean_order_total": [
        y_train_model.mean(),
        y_val_model.mean(),
```

```
        y_test_model.mean()
    ]
})

target_shape_check
```

	dataset	rows	missing_values	mean_order_total
0	y_train	10749	0	2253.428325
1	y_val	2304	0	1536.918902
2	y_test	2304	0	1184.834210

```
data_transformation_2_explanations = """
The target shape check confirms that y_train, y_val, and y_test conta
"""
```

```
# Do not modify this code
print_tile(size="h3", key='data_transformation_2_explanations', value=
```

data_transformation_2_explanations

The target shape check confirms that y_train, y_val, and y_test contain the correct number of rows and no missing values.

G.4 Data Transformation "<put_name_here>"

```
# Final modelling objects

X_train_model = X_train.copy()
X_val_model = X_val.copy()
X_test_model = X_test.copy()

print("Final X_train_model shape:", X_train_model.shape)
print("Final X_val_model shape:", X_val_model.shape)
print("Final X_test_model shape:", X_test_model.shape)

print("Final y_train_model shape:", y_train_model.shape)
print("Final y_val_model shape:", y_val_model.shape)
print("Final y_test_model shape:", y_test_model.shape)
```

```
Final X_train_model shape: (10749, 39)
Final X_val_model shape: (2304, 39)
Final X_test_model shape: (2304, 39)
Final y_train_model shape: (10749,)
Final y_val_model shape: (2304,)
Final y_test_model shape: (2304,)
```

```
data_transformation_3_explanations = """
Final modelling objects are created for training and evaluation. X_tr
```

```
These final objects are used in the Random Forest model training sect
"""
```

```
# Do not modify this code
print_tile(size="h3", key='data_transformation_3_explanations', value=
```

data_transformation_3_explanations

Final modelling objects are created for training and evaluation. X_train_model, X_val_model, and X_test_model contain the final input features, while y_train_model, y_val_model, and y_test_model contain the

Double-click (or enter) to edit

H. Save Datasets

> Do not change this code

```
# Do not modify this code
# Save training set
try:
  X_train.to_csv(at.folder_path / 'X_train.csv', index=False)
  y_train.to_csv(at.folder_path / 'y_train.csv', index=False)

  X_val.to_csv(at.folder_path / 'X_val.csv', index=False)
  y_val.to_csv(at.folder_path / 'y_val.csv', index=False)

  X_test.to_csv(at.folder_path / 'X_test.csv', index=False)
  y_test.to_csv(at.folder_path / 'y_test.csv', index=False)
except Exception as e:
  print(e)
```

J. Train Machine Learning Model

J.1 Import Algorithm

> Provide some explanations on why you believe this algorithm is a good fit

```
rf_model = RandomForestRegressor
rf_model
```

```
sklearn.ensemble._forest.RandomForestRegressor
def __init__(n_estimators=100, *, criterion='squared_error',
max_depth=None, min_samples_split=2, min_samples_leaf=1,
```